SESI 11 / SOFTWARE TESTING

Pengujian Otomatis dan CI/CD

Studi kasus pengajuan seller ZENTA

Bambang Herlambang | NRP 5803024019
Repositori GitHub: github.com/Bambang-code/software-testing-sesi11-automated-ci-cd

1. Aplikasi yang diuji

Aplikasi latihan Flask dan SQLite mengadaptasi model `SellerApplication` dari tugas individu Sesi 10. Pengguna dapat mengirim pengajuan dari status Draf ke Menunggu peninjauan, menarik pengajuan, lalu mengirimnya kembali. Browser mengakses aplikasi lewat HTTP lokal. Cookie sesi memisahkan pengajuan masing-masing pengguna.

2. Skenario pengujian browser

ID	Skenario	Kriteria lulus
UI-01	Kirim nama toko yang valid	Status berubah menjadi Menunggu peninjauan. Spasi di awal dan akhir nama dibuang.
UI-02 *Negatif*	Kirim nama yang hanya berisi spasi	HTTP 400, pesan validasi terlihat, dan status tetap Draf.
UI-03	Muat ulang halaman setelah mengirim	Nama toko dan status Menunggu peninjauan tetap terlihat di sesi yang sama.
UI-04	Tarik pengajuan dan kirim ulang	Status kembali ke Draf. Pengajuan pengganti dapat dikirim.
UI-05	Buka formulir dari konteks browser lain	Sesi kedua dimulai dari Draf tanpa menampilkan pengajuan sesi pertama.

Dua tes integrasi HTTP memeriksa respons 409 untuk kiriman ganda dan penarikan tanpa pengajuan. Sebanyak 82 tes unit memeriksa model pengajuan seller.

3. Cara menjaga tes tetap stabil

Page Object `SellerPage` memusatkan locator `data-testid` dan aksi formulir. Assertion `expect(...).to_have_text(...)` menunggu keadaan yang diharapkan, sedangkan tes negatif menunggu respons HTTP yang dituju. Tiap tes browser membuka konteks baru dengan cookie terpisah. Suite menggunakan database SQLite sementara.

4. Pipeline CI dan hasil uji

GitHub Actions / ZENTA CI

Setiap push dan pull request memicu workflow. Runner Ubuntu menyiapkan Python 3.10 dan Chromium untuk suite lengkap, lalu menjalankan PyTest dengan `--cov-branch`. Gerbang kualitas mensyaratkan cakupan gabungan statement dan tujuan cabang minimal 80%.

Hasil eksekusi lokal

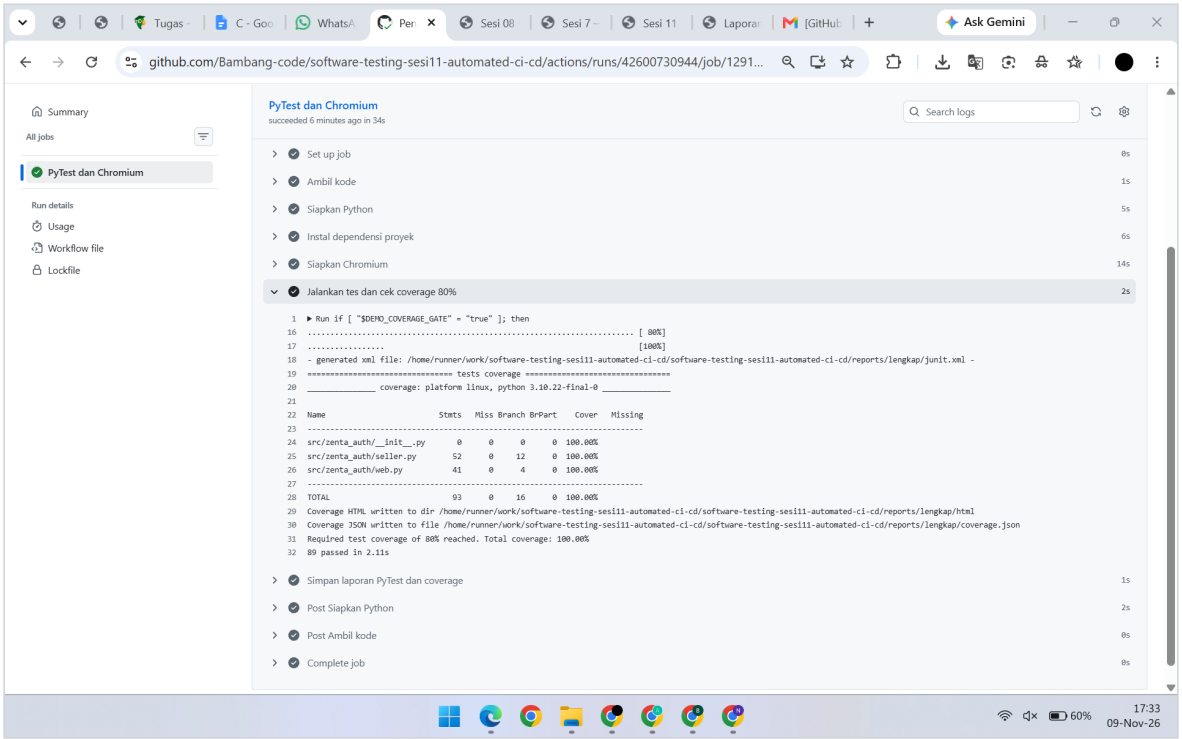
Mode	Tes lulus	Statement	Tujuan cabang	Coverage	Kode keluar
Lengkap	89	93/93	16/16	**100,00%**	0
Demo gagal	82	52/93	12/16	**58,72%**	1

Dalam mode demo, hanya tes unit yang dijalankan sementara pengukuran tetap mencakup modul web. Akibatnya, 41 statement dan empat tujuan cabang luput. Gerbang kualitas menghentikan proses meskipun semua tes unit lulus.

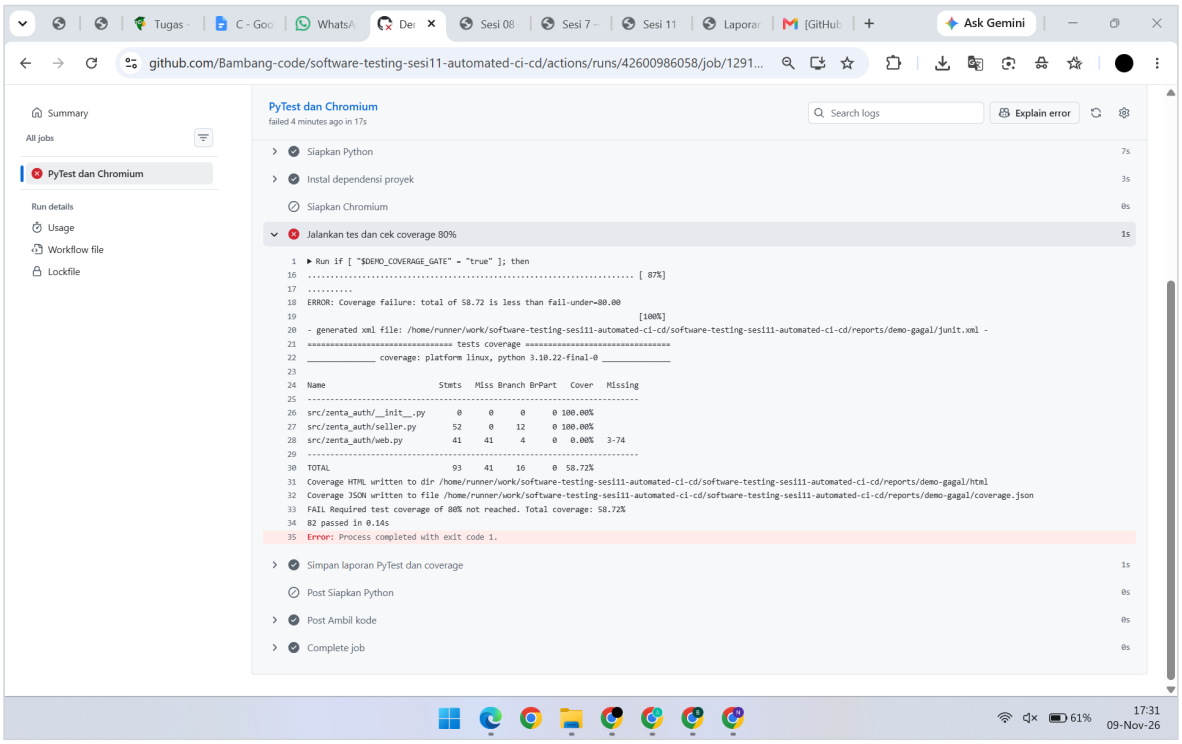
Folder `reports/lengkap/` dan `reports/demo-gagal/` berisi log PyTest, JUnit XML, coverage JSON, dan laporan HTML. Di GitHub Actions, hasil diunggah sebagai `zenta-test-reports` bahkan saat langkah tes gagal.

5. Bukti GitHub Actions

Tangkapan layar diambil setelah kode masuk ke branch default repo GitHub pribadi. Angka pada tabel di atas berasal dari pengujian lokal.



Gambar 1. Run lulus: job PyTest dan Chromium berhasil dalam 34 detik. Langkah tes melaporkan 89 tes lulus dan coverage 100,00%, di atas ambang 80%.



Gambar 2. Run sengaja gagal: langkah tes berhenti dengan kode keluar 1. Coverage total 58,72%, di bawah ambang 80%, meskipun 82 tes unit lulus.

Analisis singkat

Kedua run memakai workflow ZENTA CI yang sama, dan angkanya sama dengan hasil lokal pada bagian 4. Pada run lulus, 89 tes lulus dan coverage mencapai 100,00%, melampaui ambang 80%, sehingga job berstatus berhasil.

Pada run demonstrasi, langkah Siapkan Chromium dilewati dan hanya 82 tes unit yang berjalan. Berkas `web.py` tidak tereksekusi sama sekali (41 dari 41 statement luput, baris 3-74), sehingga total coverage turun ke 58,72%. Proses berakhir dengan kode keluar 1 dan job berstatus gagal meskipun seluruh tes unit lulus. Ini menunjukkan bahwa gerbang kualitas bekerja. Langkah penyimpanan laporan tetap berjalan, jadi bukti kegagalan tetap tersimpan.

Acuan

Materi kuliah *Sesi 11 - Automated Testing & Pengenalan CI_CD*, hlm. 8.

Kode dasar tugas Sesi 10 | GitHub Actions | Playwright CI